FIAP

GRADUAÇÃO

45697056

ES
Desenvolvimento Mobile

Prof. Adeilton Meneses

profadeilton.meneses@fiap.com.br

Introdução aos Hooks no React Native

45697056

- **O que são Hooks?**
  - Hooks são funções especiais no React que permitem que você "engate" no estado e no ciclo de vida de um componente de função. Antes dos Hooks, essas funcionalidades estavam disponíveis apenas em componentes de classe.

Documentação: https://react.dev/reference/react/hooks

Introdução aos Hooks no React Native

45697056

- **O que são Hooks?**
  - Com a introdução dos Hooks na versão 16.8 do React, tornou-se possível escrever componentes totalmente funcionais que gerenciam seu próprio estado e efeitos colaterais (como requisições a APIs), tornando o código mais conciso e reutilizável.
  - Resolver a complexidade dos componentes de classe, o compartilhamento de lógica com `render props` e `higher-order components`.

Documentação: https://react.dev/reference/react/hooks

Introdução aos Hooks no React Native

45697056

- **Por que usar Hooks?**

  Os Hooks trazem várias vantagens:

  - **Reutilização de lógica com estado**: Com Hooks personalizados (*custom hooks*), você pode extrair a lógica de estado de um componente e reutilizá-la em vários outros.
  - **Código mais limpo**: Eles ajudam a organizar a lógica relacionada ao estado em um só lugar, em vez de espalhá-la por diferentes métodos de ciclo de vida, como acontece em componentes de classe.
  - **Componentes de função**: Permitem que você use o React sem a necessidade de componentes de classe, que são mais complexos de entender e usar.
  - **Simplificação do código**: Elimina a necessidade de classes e do uso de `this`.

Hooks Essenciais

45697056

- *useState*
- *useEffect*
- *useContext*
- *Criar seu Hook (Custom Hook)*

Hooks Essenciais - *useState*

45697056

- **Conceito:** O Hook mais fundamental. Ele adiciona o estado a um componente funcional.

- **Sintaxe:** **const [state, setState] = useState(initialState);**
  - **state**: A variável de estado.
  - **setState**: A função para atualizar o estado.
  - **initialState**: O valor inicial do estado.

Exemplos Práticos

45697056

Gerenciador de Tarefas (useState)

- **Objetivo:** Construir uma lista de tarefas.

- **Requisitos:**
  - Um **TextInput** para adicionar novas tarefas.
  - Um botão para adicionar a tarefa à lista.
  - Exibir a lista de tarefas em um **FlatList**.
  - Usar **useState** para gerenciar a lista de tarefas e o texto de entrada.

Hooks Essenciais - *useEffect*

- **Conceito:** Permite realizar efeitos colaterais em componentes funcionais. O que são efeitos colaterais? Requisições de API, timers, eventos, etc.

- **Sintaxe:** **useEffect(callback, [dependencies]);**
  - callback: A função que será executada.
  - [dependencies]: Opcional. Uma lista de variáveis. O efeito será executado apenas se alguma delas mudar.

- **Exemplo Prático (Requisição de API):**
  - O useEffect é usado para fazer a chamada à API quando o componente é montado.
  - Importância do array de dependências vazio ([]) para garantir que a requisição seja feita apenas uma vez.

Hooks Essenciais - *useEffect*

45697056

Com Array de Dependências Vazio [ ]

Este é o caso mais comum para efeitos que devem ser executados apenas uma vez, quando o componente é montado. Pense nisso como o substituto do `componentDidMount` das classes. A função de limpeza (retorno) é executada quando o componente é desmontado (`componentWillUnmount`).

Exemplo: Requisição de API

```
useEffect(() => {
  // Simulando uma chamada de API
  const fetchProducts = async () => {
    try {
      const response = await fetch('https://fakestoreapi.com/products');
      const data = await response.json();
      setProducts(data);
    } catch (error) {
      console.error('Erro ao buscar produtos:', error);
    } finally {
      setLoading(false);
    }
  };

  fetchProducts();
}, []); // O array vazio garante que o efeito só rode na montagem do componente.
```

Hooks Essenciais - *useEffect*

45697056

Sem Array de Dependências

Se você não passar o segundo argumento (o array de dependências), o `useEffect` será executado após **cada renderização** do componente. Isso inclui a montagem inicial e qualquer atualização de estado ou prop. Use com cautela, pois pode causar loops infinitos se você atualizar o estado dentro do efeito.

Exemplo: Contador

```
useEffect(() => {
  // Este efeito roda a cada vez que o 'count' muda,
  // ou qualquer outro estado ou prop do componente.
  console.log(`O contador foi atualizado para: ${count}`);
}); // Sem array de dependências.
```

Hooks Essenciais - *useEffect*

45697056

Com Parâmetros no Array de Dependências `[x, y]`

Este é o caso para efeitos que precisam ser reexecutados quando valores específicos mudam. O `useEffect` será acionado na montagem do componente e sempre que qualquer variável dentro do array de dependências for alterada.

Exemplo: Busca Dinâmica com `id`

```
useEffect(() => {
  const fetchUser = async () => {
    setLoading(true);
    try {
      const response = await fetch(`https://jsonplaceholder.typicode.com/users/${us
      const data = await response.json();
      setUser(data);
    } catch (error) {
      console.error('Erro ao buscar usuário:', error);
    } finally {
      setLoading(false);
    }
  };

  // Este efeito só será executado quando a 'prop' 'userId' mudar.
  if (userId) { // Adiciona uma verificação para evitar a busca inicial com 'null'
    fetchUser();
  }

}, [userId]); // O efeito roda quando o `userId` muda.
```

Exemplos Práticos

45697056

Carregamento de Dados (`useEffect`)

- **Objetivo:** Simular o carregamento de dados de um servidor.

- **Requisitos:**
  - Usar `useEffect` para simular uma chamada assíncrona que retorna uma lista de "produtos" após 2 segundos.
  - Usar `useState` para gerenciar o estado de carregamento (`isLoading`) e a lista de produtos.
  - Exibir um texto "Carregando..." enquanto os dados não chegam e, depois, exibir a lista de produtos.

Hooks Essenciais - *`useContext`*

45697056

- **Conceito:** Uma maneira de passar dados por toda a árvore de componentes sem ter que passar as props manualmente em cada nível (**prop drilling**).

- **Sintaxe:** **const contextValue = useContext(MyContext);**

- **Exemplo Prático (Tema da Aplicação):**
  - Criar um contexto para gerenciar o tema da aplicação (claro ou escuro).
  - Demonstrar como o **useContext** é usado em um componente aninhado para acessar o valor do tema e alterná-lo.

Exemplos Práticos

45697056

Login e Autenticação (`useContext`)

- **Objetivo:** Gerenciar o estado de autenticação do usuário.

- **Requisitos:**
  - Criar um `AuthContext` que armazena o status de `isLoggedIn` e o nome do usuário.
  - Criar um componente `LoginScreen` que, ao clicar em um botão, atualiza o `AuthContext` para `isLoggedIn: true`.
  - Criar um componente `HomeScreen` que usa o `useContext` para exibir uma mensagem de boas-vindas com o nome do usuário se ele estiver logado. Caso contrário, exibe um botão para ir para a tela de login.

Hooks Customizado

45697056

O que é?

- Um **hook customizado** no React é uma função cujo nome começa com "use" e que pode chamar outros hooks, como **useState** ou **useEffect**. A principal finalidade de um hook customizado é permitir que você extraia a lógica de um componente para uma função reutilizável.

- Imagine que você tem vários componentes que precisam realizar a mesma tarefa, como buscar dados de uma API, controlar o estado de um formulário ou gerenciar a largura da tela. Em vez de duplicar o código em cada componente, você pode criar um hook customizado que encapsula essa lógica.

Hooks Customizado

45697056

Como usá-lo?

Para usar um hook customizado, você precisa primeiro defini-lo. A convenção é que o nome da função comece com ***use***. Dentro da função, você pode usar os hooks nativos do React e qualquer outra lógica que precise.

Depois de definido, você pode simplesmente chamar o hook customizado dentro de um componente de função React, como se estivesse usando qualquer outro hook.

Exemplos Práticos

45697056

Definir um título (`Hook customizado`)

```
import { useEffect } from 'react';

// O nome do hook começa com 'use'
function useDocumentTitle(title) {
  useEffect(() => {
    // Isso é o que a gente quer fazer: atualizar o título do documento
    document.title = title;

    // A função de "cleanup" é opcional, mas boa prática.
    // Ela garante que, quando o componente for desmontado,
    // o título volte a ser o original (ou outro valor padrão).
    return () => {
      document.title = 'React App'; // Título padrão
    };
  }, [title]); // O array de dependências garante que o efeito só rode quando o 't
}

export default useDocumentTitle;
```

Exemplos Práticos

45697056

Definir um título (`Hook customizado`)

```
// Exemplo em um componente de página
import React from 'react';
import useDocumentTitle from './useDocumentTitle'; // Importe o hook

function PaginaDeProduto() {
  // Chame o hook e passe o título que você quer
  useDocumentTitle('Detalhes do Produto');

  return (
    <div>
      <h1>Página do Produto</h1>
      <p>Aqui você encontra todas as informações sobre o nosso produto.</p>
    </div>
  );
}

export default PaginaDeProduto;
```

Vamos conhecer o código?

Criando projeto

45697056

- Criando nosso projeto React native + Expo
  - npx create-expo-app --template
  - selecionar template Blank (Typescript)
  - definir nome do projeto
  - entrar na pasta criada (cd "nome da pasta")
  - npm start
  - escolher plataforma
    - **A** - android
    - **I** - iOS

Explicação comandos

45697056

npx create-expo-app - -template

- Criar um novo projeto Expo usando o template blank ou de sua preferência.
- npx: executa pacotes sem precisar instalá-los globalmente.
- create-expo-app: ferramenta para iniciar projetos Expo.
- –template: lista templates (com React Native + Expo).

45697056

Projeto base

https://github.com/admeneses/react-hooks-initial

Como executar a primeira vez:

- clonar o projeto
- npm i

Depois de configurado:

npm start + (a plataforma que será executado)

Obrigado

2025-Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proíbido sem o consentimento formal, por escrito, do Professor Wellington Cidade Silva Tenorio.